Adaptive Feature Self-Attention in Spiking Neural Networks for Hyperspectral Classification

Heng Li, *Graduate Student Member, IEEE*, Bing Tu, *Senior Member, IEEE*, Bo Liu, *Member, IEEE*, Jun Li, *Fellow, IEEE*, and Antonio Plaza, *Fellow, IEEE*

***Abstract*— Hyperspectral image (HSI) classification is crucial for remote sensing research, while its high-dimensional features make traditional algorithms difficult to cope with. Despite the breakthroughs in deep learning, the high computational complexity and energy consumption limit its application in resource-limited environments. Spiking neural networks (SNNs), mimicking the brain's information processing with low power consumption, have emerged as a promising alternative for edge computing. However, SNNs struggle with complex tasks due to the nondifferentiability of spike signals, which complicates training and exhibits limitations in extracting deep features and modeling long-range dependencies. In this article, we propose a novel SNN framework that addresses these challenges by enhancing feature extraction and efficiently capturing dependencies in hyperspectral data. Our framework integrates an adaptive refocusing convolutional layer with a spike self-attention (SSA) mechanism. The adaptive refocusing convolutional layer employs learnable parameters to dynamically adjust the convolutional kernel's response to input spike data, improving feature representation. The adaptive refocusing convolutional layer uses learnable parameters to dynamically adjust kernel responses to input spike data, enhancing feature representation. Experimental results show that this model achieves over 96% classification accuracy in a single time step, significantly surpassing current methods and effectively solving the problem of low accuracy at short time steps in SNNs. Additionally, this framework reduces computational energy consumption by approximately 12.5× compared to similar, offering new potential for edge intelligence applications.**

***Index Terms*— Convolutional neural networks (CNNs), hyperspectral image (HSI) classification, spike self-attention (SSA), spiking neural networks (SNNs).**

Received 17 August 2024; revised 28 October 2024; accepted 9 December 2024. Date of publication 13 December 2024; date of current version 24 December 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62271200 and in part by the Start-Up Foundation for Introducing Talent of Nanjing University of Information Science and Technology (NUIST) under Grant 2023r091. *(Corresponding author: Bing Tu.)*

Heng Li, Bing Tu, and Bo Liu are with the School of Physics and Optoelectronic Engineering, Institute of Optics and Electronics, State Key Laboratory Cultivation Base of Atmospheric Optoelectronic Detection and Information Fusion, Jiangsu International Joint Laboratory on Meteorological Photonics and Optoelectronic Detection, Jiangsu Engineering Research Center for Intelligent Optoelectronic Sensing Technology of Atmosphere, Nanjing University of Information Science and Technology, Nanjing, Jiangsu 210044, China (e-mail: 202312490732@nuist.edu.cn; tubing@nuist.edu.cn; bo@nuist.edu.cn).

Jun Li is with the Faculty of Computer Science, China University of Geosciences, Wuhan 430074, China (e-mail: lijuncug@cug.edu.cn).

Antonio Plaza is with the Hyperspectral Computing Laboratory, Department of Technology of Computers and Communications, Escuela Politecnica, University of Extremadura, 10003 Cáceres, Spain (e-mail: aplaza@unex.es).

Digital Object Identifier 10.1109/TGRS.2024.3516742

I. Introduction

HYPERSPECTRAL images (HSIs) are captured with high precision, recording a continuum of wavelengths ranging from the visible to the near-infrared spectrum. These images are characterized by their extensive spectral resolution, covering a range from tens to hundreds of spectral bands [1], [2]. Each pixel in an HSI represents the physical characteristics and reveals the intricate chemical properties of the observed objects. Classifying these images is essential for various applications, including agricultural monitoring, land use analysis, urban surveillance, mineral exploration, and ecological studies [3], [4].

Traditional methods for classifying HSIs, such as maximum likelihood classification (MLC) [5], [6], support vector machines (SVMs) [7], and random forest (RF) [8] are commonly used but often fail to capture the spatial dynamics of the scenes. These techniques generally struggle with challenges such as spectral mixing and spatial discontinuities, which can impair classification accuracy. To address these issues, object-based classification approaches have been increasingly used, improving feature extraction through methods such as principal component analysis (PCA) [9] and linear discriminant analysis (LDA) [10].

The field of HSI classification was significantly advanced by the introduction of deep learning, especially through the integration of models like convolutional neural networks (CNNs), recurrent neural networks (RNNs), deep neural networks (DNNs), generative adversarial networks (GANs) [11], [12], and graph convolutional networks (GCNs) [13], [14], [15]. These technologies greatly improved both the efficiency and accuracy of classification by effectively addressing challenges like handling high-dimensional data and correcting sample imbalances. Additionally, these models were refined to enhance their capability for integrating features. Despite these advancements, the models posed challenges, primarily due to high computational and energy demands. This issue was especially problematic in scenarios requiring real-time and edge computing [16], [17], such as UAV monitoring and satellite image processing, where efficient resource usage was critical [18], [19].

In contrast, spiking neural networks (SNNs) have emerged as a promising solution in resource-constrained environments due to their low energy consumption and biologically inspired computational framework [20]. These networks, replicating the impulse-firing mechanism of biological neurons [21], process

1558-0644 © 2024 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 11:04:01 UTC from IEEE Xplore. Restrictions apply.

information through discrete pulse sequences, achieving high energy efficiency and parallel processing capabilities [22], [23], [24]. However, traditional backpropagation (BP) algorithms have been less suitable for SNNs due to the discrete nature of pulse signals, making it challenging to encode continuous hyperspectral data into these sequences.

To address these limitations, Lee et al. [25] proposed an approximate derivative method to train DNN architectures via simulated spike BP, mitigating the leakage behavior in LIF neurons and facilitating the conversion of trained artificial neural networks (ANNs) into SNNs. Wunderlich and Pehle [26] introduced an event-based BP algorithm capable of computing the exact gradient in SNNs, thereby overcoming challenges associated with discrete spiking events and discontinuities. Zhang and Li [27] developed the spike-train-level recurrent SNN BP (ST-RSBP) algorithm, utilizing specific loss functions to improve the supervised training of deep RSNNs.

Regarding optimization strategies, Kulkarni and Rajendran [28] explored methods to improve accuracy by refining learning parameters and network configurations, proposing strategies for deploying SNNs on memory- and energy-constrained hardware. Tolooshams et al. [29] introduced constrained recurrent sparse autoencoders (CRsAEs) for convolutional dictionary learning, creating a connection between dictionary learning and neural networks. Wang et al. [30] proposed an incremental agent gradient learning method to approximate the step function and reduce information loss, improving the BP process in SNNs. Additionally, Wu et al. [31] developed a tandem learning framework that combines SNNs and ANNs through weight sharing, offering a novel approach for training efficient, low-latency, and high-precision deep SNNs under limited computational resources. Chen et al. [32] introduced a high-performance SNN with spike frequency adaptation (SFA-SNN), which enhances image classification performance by integrating a dynamic discharge threshold into the neuronal membrane potential equation.

In terms of performance enhancement, the weight normalization method proposed by Xing et al. [33] reduces the performance gap between converted SNNs and original CNNs. By adjusting the weight distribution, this method allows the pulse-firing rate of SNNs to more closely approximate the activation values of CNNs, providing a novel approach for achieving high-performance SNNs. The successful application of the SNN-CNN hybrid network developed by Zhang et al. [34] in autopilot systems highlights the potential of integrating these networks, which is anticipated to enhance SNN performance in complex visual tasks while maintaining low energy consumption. Additionally, several studies have explored the integration of SNNs and CNNs on FPGAs [35], [36], offering new possibilities for neural network hardware acceleration. These advancements not only improve model execution efficiency but also lay the groundwork for the deployment of SNNs in edge computing devices.

In remote dependency modeling, Zhou et al. [37] are the first to combine SNNs with Transformer architectures, introducing the impulse attention module [spike self-attention (SSA)] and pioneering a new research direction [38], [39], [40]. Inspired by this innovation, subsequent studies have further explored the integration of SNNs and Transformers,

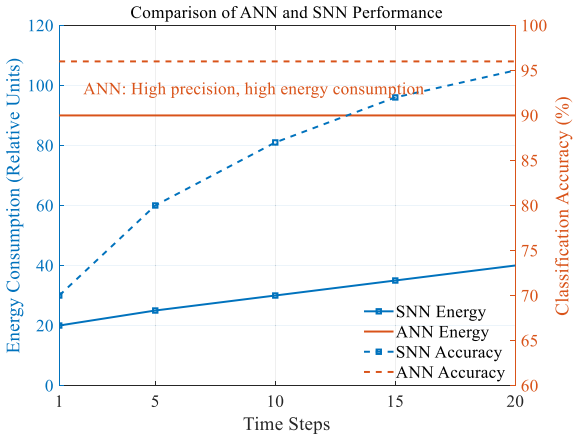


Fig. 1. Comparative Analysis of ANN and SNN in terms of energy consumption and classification accuracy over time steps.

enhancing SNNs' capabilities in processing long-sequence data and providing new avenues for designing more efficient neural network architectures. To address the challenges of SNN training, researchers have developed various advanced learning algorithms, such as alternative gradient-based BP [41] and time-coded supervised learning methods [42], significantly improving the training efficiency and performance of SNNs. Despite these advancements, challenges remain in applying SNNs to complex tasks like HSI classification.

For instance, the hyperspectral classification framework proposed by Liu et al. [43], while generally effective, still exhibits deficiencies in classification performance at shorter time steps. As shown in Fig. 1, spiking neural network (SNN) exhibit lower energy consumption compared to artificial neural network (ANN), but their performance under short time steps remains limited. In order to address the classification accuracy of SNNs under short time steps in HSI classification tasks, as well as the challenges in feature extraction and dependency modeling, this article proposes a focused convolutional SNN framework incorporating an attention mechanism. The contributions of this article are summarized as follows.

1) We propose an adaptive refocusing convolutional layer, which introduces learnable refocusing parameters that can adaptively adjust the output of the convolutional layer according to the characteristics of the input pulse signal. With this dynamic adjustment mechanism, the model is able to capture key features in high-dimensional hyperspectral data more effectively, enhancing the accuracy and flexibility of feature extraction, and thus improving the adaptability and performance of SNNs in complex tasks.
2) In order to compensate for the shortcomings of SNNs in long-range and mid-range dependency modeling, this article introduces the impulse self-attention mechanism, which calculates the attention weights by accumulating membrane potentials. This mechanism can effectively capture the long-range dependencies between spectral and spatial features in HSIs, thereby strengthening the SNN's ability to express complex spatio-temporal features and significantly improving classification performance.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 11:04:01 UTC from IEEE Xplore. Restrictions apply.

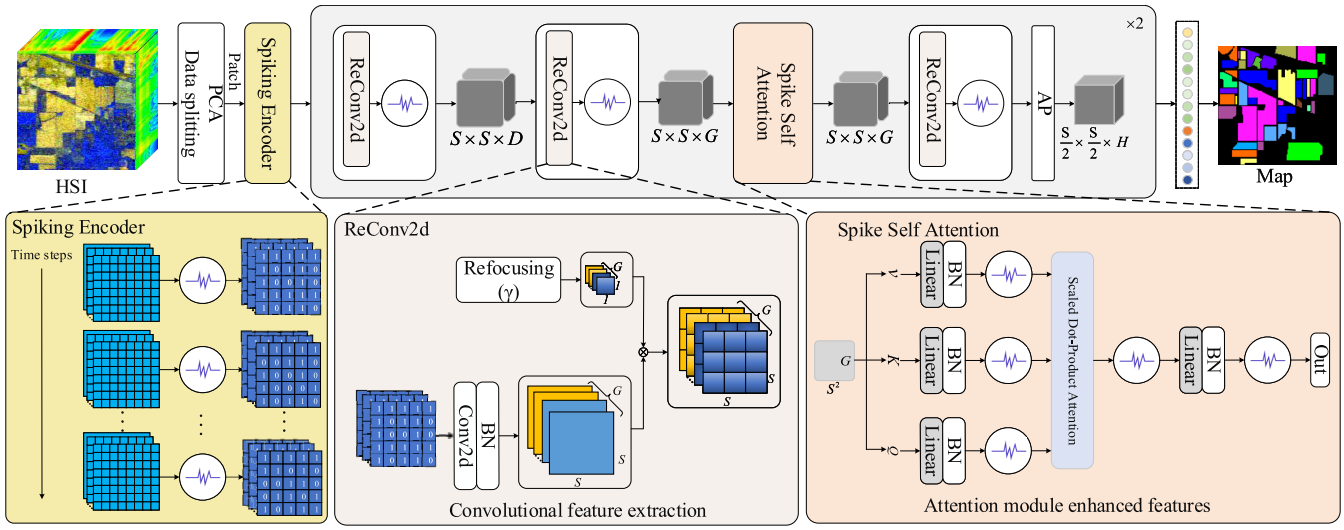


Fig. 2. Framework of the proposed model (advanced SNN framework utilizing direct encoding, deep convolutional layers, and robust attention mechanisms for enhanced accuracy in HSI classification).

3) We integrate an adaptive refocusing convolutional layer with a SSA mechanism into a unified SNN framework to achieve efficient feature extraction and dependency modeling. Experimental results show that the framework achieves more than 96% classification accuracy at short time steps, which is significantly better than existing methods, and reduces energy consumption by about 12.5×, demonstrating its great potential for application in resource-constrained environments.

The remainder of the article is organized as follows. Section II outlines the methodologies and architectural details for constructing the proposed framework. Section III discusses the optimal classification parameters identified through empirical analysis across four hyperspectral datasets. Section IV presents a comprehensive comparison with other methods. Section V provides an analysis of the proposed method's energy consumption. Finally, Section VI concludes the article and suggests future research directions.

II. Methodology

A. Background and Fundamentals

1) Neuron Model: In the human nervous system, neurons integrate excitatory and inhibitory inputs, gradually raising their membrane potential until it reaches a threshold, triggering an action potential. After firing, the neuron enters a refractory period, temporarily unable to respond to new inputs, while the membrane potential gradually decays. This action potential is then transmitted to downstream neurons via synapses. Inspired by these processes, computational models such as Hodgkin–Huxley, Izhikevich, and adaptive exponential (AdEx) have been developed [44], [45]. The leaky integrate-and-fire (LIF) model provides a simplified yet effective representation of these neuronal dynamics [46].

The LIF neuron model focuses on the dynamics of membrane potential. The fundamental principle is that the input signals received by the neuron gradually increase its membrane potential. Once this potential reaches a specific threshold, the neuron emits a spike, after which the membrane potential is reset. The governing equation for the membrane potential is given by

$$\tau_m \frac{dV(t)}{dt} = -V(t) + RI(t) \tag{1}$$

where τ_m denotes the membrane time constant, $V(t)$ represents the membrane potential at time t, R denotes the resistance, and $I(t)$ represents the input current at time t.

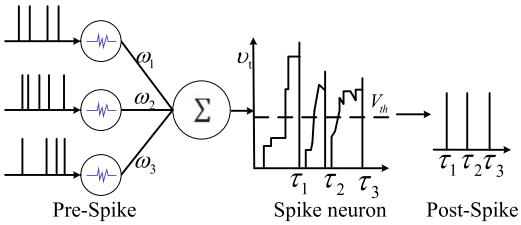


Fig. 3. Dynamics of the LIF neuron model.

As shown in Fig. 2, the framework proposed in this paper uses the improved LIF model adopted by the predecessors, which can accumulate more spike potential signals in a shorter time step. The modified formula for the membrane potential $V(t)$ of the LIF neuron is presented below. The forward propagation process of the LIF neuron model is illustrated in Fig. 3

$$V_i^t = \beta V_i^{t-1} + \sum_j W_{ij} S_j^{t-1} \tag{2}$$

where β represents the membrane potential decay constant, ranging between 0 and 1. V_i^t denotes the membrane potential of neuron i at time step t. W_{ij} denotes the synaptic weight from neuron j to neuron i. S_j^{t-1} represents the output pulse of neuron j at time step $t-1$. The update equation for the membrane potential at each time step is as follows:

$$V_i^t = \beta V_i^{t-1} + I_i^t - \Theta \tag{3}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 11:04:01 UTC from IEEE Xplore. Restrictions apply.

where Θ denotes the threshold potential. The condition for generating output pulses is as follows:

$$S_i^t = \begin{cases} 1, & \text{if } V_i^t \geq \Theta \\ 0, & \text{if } V_i^t < \Theta. \end{cases} \tag{4}$$

When a neuron releases a pulse signal, its membrane potential resets to the resting state until the next pulse signal arrives. Neurons in the final layer of the network receive all forward-propagated spike signals. The final output (voltage) is obtained by dividing the membrane potential V by the total number of time steps

$$\text{Output} = \frac{1}{T}\sum_{t=1}^{T} V_i^t. \tag{5}$$

2) Approximate Function: In order to effectively optimize the network parameters, this model defines a loss function aimed at minimizing the discrepancy between the predicted output and the true label. The weights of each layer are updated using the chain rule of differentiation. The derivative of the weight parameter w is calculated as follows:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial S}\frac{\partial S}{\partial V}\frac{\partial V}{\partial w} \tag{6}$$

where L represents the loss function, w denotes the network weight parameter, V is the neuron's membrane potential, and S corresponds to the output spike. However, due to the discrete nature of spike signals, their gradients cannot be directly computed. This nondifferentiability in SNNs poses a challenge for traditional BP, as the derivative of the spike output is not easily obtainable.

Additionally, as highlighted in [45], the dynamic and binary characteristics of spike signals can lead to vanishing or exploding gradients, particularly in deep networks, hampering training efficiency and convergence.

To overcome these issues, we introduce a differentiable approximation function to replace the discrete derivative of the spike activation. The chosen approximation function is

$$f(v) = \max(0, 1 - v^2). \tag{7}$$

The proposed approximation $f(v) = \max(0, 1 - v^2)$ offers several advantages. First, this function is smooth and continuous in the interval $-1 < v < 1$, which ensures that a nonzero gradient is provided when v is close to zero. Specifically, the derivative of this function is

$$\frac{\partial f(v)}{\partial v} = -2v, \quad \text{for } -1 < v < 1. \tag{8}$$

For values of $|v| \geq 1$, the derivative becomes 0, indicating that the function effectively ignores values of the membrane potential that are far from the spiking threshold. This behavior is beneficial in preventing unnecessary gradient calculations, reducing computational overhead, and focusing the learning process on relevant spike activities.

The smooth gradient provided by $f(V)$ enables efficient gradient propagation during BP, allowing the training algorithm to update weights even for signals that would traditionally be considered nondifferentiable. This characteristic helps to mitigate the issues of vanishing and exploding gradients, as it ensures that neurons with membrane potentials near the spike threshold continue to contribute to the overall gradient.

By employing the approximate derivative function $f(v) = \max(0, 1 - v^2)$, the SNN can facilitate gradient propagation, thus supporting efficient training and optimization. Specifically, given the loss function L, the gradient calculation formula for the weight parameters across all time steps is

$$\frac{\partial L}{\partial w} = \sum_{t=1}^{T} \frac{\partial L}{\partial S^t}\frac{\partial S^t}{\partial V^t}\frac{\partial V^t}{\partial w}. \tag{9}$$

Since S is a discrete spike output, the direct derivative $(\partial S/\partial V)$ does not exist. Hence, we substitute it with the approximate derivative

$$\frac{\partial S}{\partial V} \approx f(V) = \max(0, 1 - V^2). \tag{10}$$

This substitution smooths gradient computation across time steps, ensuring that spikes contribute to learning despite their discrete activation. The function $f(v) = \max(0, 1 - v^2)$ preserves key spike signal characteristics, providing high gradient sensitivity when the membrane potential V is near the spiking threshold. This allows small changes in V to produce significant gradient updates, crucial for SNNs' responsiveness to sparse, event-driven spikes. Additionally, $f(v) = \max(0, 1 - v^2)$ is computationally efficient, reducing unnecessary calculations for values of V far from the threshold, which supports energy-efficient training—a key benefit of SNNs.

B. Spike Encoding

The spike encoding layer is an essential part of the model, converting continuous image data into pulse signals compatible with SNNs. This conversion allows the SNN to operate in an event-driven manner, improving efficiency and conserving energy. To enable effective training, a surrogate BP function is employed to handle the nondifferentiability of pulse signals, enabling gradient-based optimization.

For HSI classification, the SNN transforms the input image into a binary spike sequence using the spike encoding layer and processes it iteratively across multiple time steps. The two primary spike encoding schemes are rate coding and direct coding.

Rate coding, as illustrated in Fig. 4, is a biologically inspired encoding scheme that uses the firing frequency of neurons to represent the intensity of the input signal. In this scheme, the signal intensity is directly proportional to the neuron's firing frequency, meaning that a higher input intensity results in a higher firing rate. The relationship between the firing frequency and signal intensity can be expressed as

$$f = k \cdot I \tag{11}$$

where f represents the firing frequency, I denotes the input signal intensity, and k is a proportional constant. While rate coding closely mimics biological neuron behavior, it often requires many time steps to accurately convey intensity information, leading to increased computational complexity and energy consumption. This makes rate coding less efficient for tasks like HSI classification, where rapid inference is needed.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 11:04:01 UTC from IEEE Xplore. Restrictions apply.

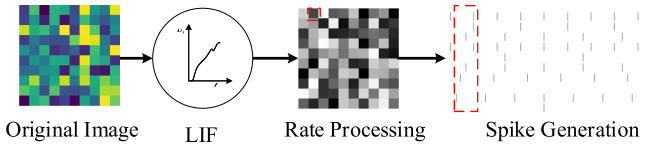


Fig. 4. Rate encoding model in SNNs.

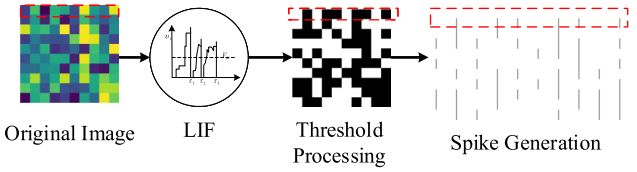


Fig. 5. Direct encoding model in SNNs.

In contrast, direct coding, shown in Fig. 5, provides a more efficient method by integrating input images to generate floating-point outputs. These outputs are compared to preset thresholds: if the output exceeds the threshold, the neuron fires a spike; otherwise, it remains silent. The encoding process is mathematically expressed as

$$S = \text{sign}(V - \Theta) \tag{12}$$

where S is the output spike, V represents the integrated input signal, and Θ denotes the threshold. Direct coding provides significant advantages, such as faster response times and reduced computational complexity, as it does not require multiple time steps to represent signal intensity. This allows the SNN to capture essential information from input data in just one or a few time steps, making it ideal for applications requiring quick decision-making.

A recent study [47] compared rate coding and direct coding in terms of classification accuracy, noise robustness, and energy efficiency. The results showed that direct coding achieved higher accuracy with fewer time steps, greater noise robustness, and more efficient spike generation, leading to lower energy consumption—critical in resource-constrained environments.

Our experimental observations also highlighted key advantages of direct coding.

1) Direct coding captures essential features in HSIs, offering superior classification with fewer time steps.
2) Unlike rate coding, direct coding encodes input data in a single step, enabling faster inference and training with lower computational costs.
3) Fewer time steps and reduced spike activity significantly lower energy consumption, making it ideal for edge computing.
4) The thresholding mechanism naturally filters out noise, ensuring more stable and accurate encoding.

Given these advantages, we adopted direct coding for our spike encoding layer. As illustrated in Figs. 4 and 5, direct coding accelerates spike generation and ensures efficient feature capture, contributing to the overall effectiveness and efficiency of our model in HSI classification.

C. Refocusing Convolution

In conventional CNNs, fixed convolutional kernels generate feature maps effectively for standard tasks but struggle with complex, high-dimensional datasets like HSIs. The static nature of these kernels limits adaptability in dynamic scenarios.

To address these challenges, we introduced an adaptive refocusing convolutional layer for SNNs. This layer uses a vector of learnable refocusing parameters, α, allowing the network to dynamically adjust feature intensity across each channel after convolution. This enhances the network's ability to adapt to diverse and changing data environments

$$Y = W * X + b \tag{13}$$

where Y represents the output feature map after convolution, W denotes the convolution kernel weight, X represents the input feature map, and b is the bias term. Next, Y undergoes batch normalization (BN) to stabilize the learning process and enhance the network's generalization ability. BN standardizes the features for each channel, ensuring that they have a mean of 0 and a variance of 1

$$Y_{\text{bn}} = \text{BN}(Y). \tag{14}$$

Based on this, we introduce the adaptive refocusing parameter α, which dynamically adjusts the intensity of each channel in the normalized feature map Y_{bn} according to the input features. BN plays a critical role in SNNs. Specifically, BN standardizes each channel's features to a mean of 0 and variance of 1, reducing internal covariate shift. This process stabilizes neuron activation, promotes convergence during training, and enhances the model's generalization ability. In SNNs, due to the dynamic nature of spiking neurons, fluctuations in input distribution can significantly impact membrane potential stability. By applying BN after the convolutional layer, it effectively normalizes membrane potential variations, reduces instability during training, and improves the network's response consistency and overall performance. The output feature map Y' after adaptive refocusing is given by

$$Y' = Y_{\text{bn}} \odot \alpha \tag{15}$$

where Y' represents the output feature map after adaptive refocusing, α is the learnable refocusing parameter with the same shape as the number of channels C in Y, and $\odot$ denotes element-wise multiplication (Hadamard product). The refocusing parameters enable the convolutional layer to dynamically adjust the kernel response, similar to channel-wise weighting or modulation. This allows the network to automatically determine the importance of each channel based on the training data, improving the effectiveness of feature extraction.

The adaptive refocusing parameter α dynamically adjusts each channel's feature strength based on input data, ensuring efficiency and accuracy across varying input patterns and environmental changes. This adaptive adjustment enhances the network's ability to effectively learn and extract key features, particularly beneficial for handling complex and variable data patterns like different wavelengths in HSIs.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 11:04:01 UTC from IEEE Xplore. Restrictions apply.

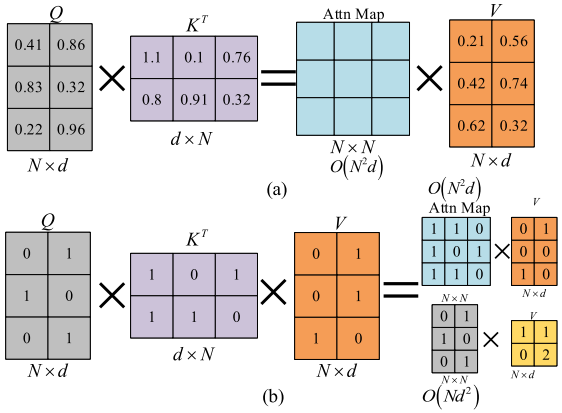


Fig. 6. Comparative analysis of computational complexity between (a) VSA and (b) SSA.

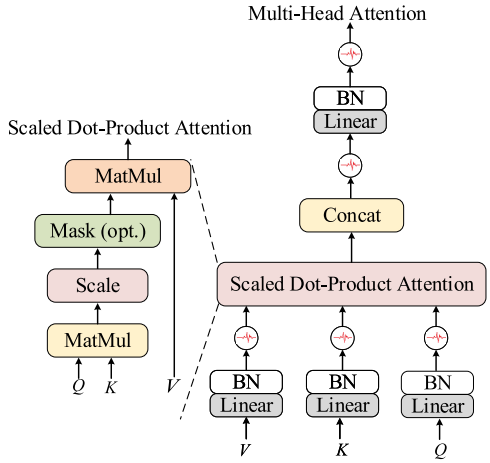


Fig. 7. Detailed operation flow of SSA: including normalization, linear layer, spike layer, and attention calculation.

D. Spike Self-Attention

The traditional self-attention mechanism, key to transformer-based models, involves complex matrix multiplications and softmax operations that demand significant computational resources. As shown in Fig. 6, the vanilla self-attention (VSA) process includes multiple matrix multiplications with the query (Q), key (K), and value (V) matrices, followed by softmax normalization to generate attention weights. This method requires $O(N^2d)$ floating-point operations (FLOPs), making it unsuitable for energy-efficient, event-driven SNNs, which use binary or multivalued discrete signals.

To address these challenges, we introduce the SSA mechanism, depicted in Fig. 7, which is tailored to the spike-based processing nature of SNNs, enabling efficient attention computation with significantly reduced complexity and energy consumption.

In SSA, the query (Q) and key (K) matrices are generated using spike-based neuron layers, making them compatible with SNNs. This transformation is defined as

$$Q = \text{spike}(XW_Q), \quad K = \text{spike}(XW_K) \tag{16}$$

where X represents the input feature matrix, W_Q and W_K are learnable weight matrices, and the "spike" function binarizes these matrices into 0 and 1 s. This binarization greatly reduces the matrix multiplication complexity, allowing for efficient computation of QK^T using simple bitwise operations.

Unlike the traditional self-attention mechanism, SSA performs attention computation directly without the softmax normalization step

$$\text{SSA}(Q, K, V) = \frac{QK^T}{\sqrt{d_k}}V. \tag{17}$$

The use of spike-based matrix multiplication allows the model to exploit data sparsity, reducing the required operations from dense FLOPs to sparse SOPs. This transition decreases computational complexity to approximately $O(Nd^2)$, making it far more efficient than traditional self-attention.

This reduction in complexity directly impacts energy consumption. As shown in Fig. 6, spike-based matrix multiplications significantly lower energy usage. This energy efficiency can be quantified by modeling the energy consumption E_{SSA}

$$E_{\text{SSA}} = \eta \cdot \text{SOPs} \tag{18}$$

where η represents the energy cost per spike operation, which is significantly lower than the energy per FLOP (ϵ) in traditional self-attention mechanisms. Thus, with $E_{\text{SSA}} \ll E_{\text{Vanilla}}$, the SSA mechanism is not only computationally efficient but also energy-efficient, making it ideal for edge computing and other resource-constrained environments. A key aspect of SNN training is efficient gradient propagation. The binary nature of SSA simplifies gradient calculation, which can be expressed as

$$\frac{\partial L}{\partial V} = \frac{\partial L}{\partial\left(\frac{QK^T}{\sqrt{d_k}}V\right)}. \tag{19}$$

Since Q and K are binary spike-encoded matrices, this ensures that the BP process remains computationally efficient. Moreover, SSA avoids the need for calculating gradients of the softmax function, further enhancing training efficiency. The derivative with respect to the input matrix X can therefore be expressed as

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial V}\frac{K^T}{\sqrt{d_k}} \tag{20}$$

where L represents the loss function. This direct gradient propagation reduces the risk of vanishing or exploding gradients, common in deep learning models, thereby improving training stability and efficiency. A key advantage of SSA is its ability to efficiently model long-range dependencies in HSI data. In high-dimensional datasets, capturing relationships across spectral bands is essential. The binary spike-based attention weights matrix in SSA, represented by $(QK^T/\sqrt{d_k})$, captures correlations between feature dimensions without requiring FLOPs

$$\text{Attention}_{\text{SSA}} = \frac{QK^T}{\sqrt{d_k}}. \tag{21}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 11:04:01 UTC from IEEE Xplore. Restrictions apply.

TABLE I
MODEL IMPLEMENTATION DETAILS

Layer Name	Input
Input Layer	HSI data block after PCA downscaling
Conv2d_1	$15 \times 15 \times 64$ (K=7, S=1)
Conv2d_2	$15 \times 15 \times 64$ (K=7, S=1)
SSA_1	$15 \times 15 \times 64$ (heads=8)
Conv2d_3	$15 \times 15 \times 128$ (K=7, S=1)
Average Pooling	$7 \times 7 \times 128$ (K=2)
Conv2d_4	$7 \times 7 \times 192$ (K=5, S=1)
Conv2d_5	$7 \times 7 \times 192$ (K=3, S=1)
SSA_2	$7 \times 7 \times 192$ (heads=12)
Conv2d_6	$7 \times 7 \times 256$ (K=3, S=1)
Average Pooling	$3 \times 3 \times 256$ (K=2)
Output Layer	Classification Maps

This enables SSA to model both spatial and spectral relationships effectively, enhancing the overall performance of the SNN by preserving crucial information while maintaining computational efficiency.

E. Framework Model

The human brain excels at performing complex tasks by receiving and emitting spikes. A neuron accumulates spikes from preceding neurons until its membrane potential surpasses a threshold, triggering a spike signal to subsequent neurons. The membrane potential also gradually dissipates over time. Inspired by these mechanisms, this article introduces a SNN framework using a direct coding scheme and LIF neuron models for hyperspectral imaging (HSI) classification.

The architecture, shown in Fig. 2 and detailed in Table I, consists of an input layer, spike encoding, refocusing convolution, SSA, average pooling, and spike decoding. PCA initially extracts key features from the HSI data, which are processed by the pulse encoder to generate a binary pulse sequence over T time steps. The sequence undergoes multiple refocusing convolutions for feature extraction.

To capture dependencies between distant pixels, the framework employs the SSA module, which uses query (Q), key (K), and value (V) vectors to enhance global contextual information. After SSA, features are further refined by pulse convolution layers, with average pooling applied to reduce spatial dimensions and parameters. Finally, the pulse decoder layer produces a pulse-voltage representation over T time steps, which is transformed into the final classification result via a fully connected layer.

III. EXPERIMENTAL SETUP AND PARAMETER DISCUSSION

A. Dataset Description

In our experiments, we utilized four HSI datasets: Indiana Pines (IP), Houston2013(HU), Pavia University (PU), and Pavia Center (PC), which were captured using AVIRIS, CASI-1500, and ROSIS sensors, respectively. Detailed sample information for these datasets is summarized in Tables II–V.

B. Parameter Setting

1) Neuronal Threshold—Impact on Network Responsiveness: To explore the effect of neuron thresholds on our proposed model in HSI classification, experiments were conducted on four datasets with thresholds set to 0.5, 0.7, 1, 1.2, and 1.5. The step size was set to 1, and the patch size was 15×15. The results, shown in Fig. 8, indicate that neuron thresholds significantly impact the performance of our proposed model. Lower thresholds (0.5, 0.7) facilitate fast information propagation but may lead to overactivation.

TABLE II
SAMPLES' INFORMATION FOR THE IP DATASET

IP dataset			
No.	Class Name	Train	Test
1	Alfalfa	37	9
2	Corn-notill	100	1328
3	Corn-mintill	100	730
4	Corn	100	137
5	Grass-pasture-mowed	100	383
6	Grass-trees	100	630
7	Grass-pasture	23	5
8	Hay-windrowed	100	378
9	Oats	16	4
10	Soybean-notill	100	872
11	Soybean-mintill	100	2355
12	Soybean-clean	100	493
13	Wheat	100	105
14	Woods	100	1165
15	Buildings	100	316
16	Stone	75	18
	Total	1351	8898

TABLE III
SAMPLES' INFORMATION FOR THE HU DATASET

HU dataset			
No.	Class Name	Train	Test
1	Healthy grass	100	1151
2	Stressed grass	100	1154
3	Synthetic grass	100	597
4	Trees	100	1144
5	Soil	100	1142
6	Water	100	225
7	Residential	100	1168
8	Commercial	100	1144
9	Road	100	1152
10	Highway	100	1127
11	Railway	100	12135
12	Parking Lot 1	100	1133
13	Parking Lot 2	100	369
14	Tennis Court	100	328
15	Running Track	100	560
	Total	1500	13529

TABLE IV
SAMPLES' INFORMATION FOR THE PU DATASET

PU dataset			
No.	Class Name	Train	Test
1	Asphalt	100	6531
2	Meadows	100	18549
3	Gravel	100	1999
4	Trees	100	2964
5	Painted metal sheets	100	1245
6	Bare soil	100	4929
7	Bitumen	100	1230
8	Self-blocking bricks	100	3582
9	Shadows	100	847
	Total	900	41876

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 11:04:01 UTC from IEEE Xplore. Restrictions apply.

TABLE V
SAMPLES' INFORMATION FOR THE PC DATASET

PC dataset			
No.	Class Name	Train	Test
1	Water	100	23043
2	Trees	100	1583
3	Asphalt	100	1766
4	Self-Blocking Bricks	100	2140
5	Bitumen	100	4012
6	Stubble	100	3837
7	Tiles	100	1728
8	Shadows	100	2088
9	Meadows	100	1672
	Total	900	41023

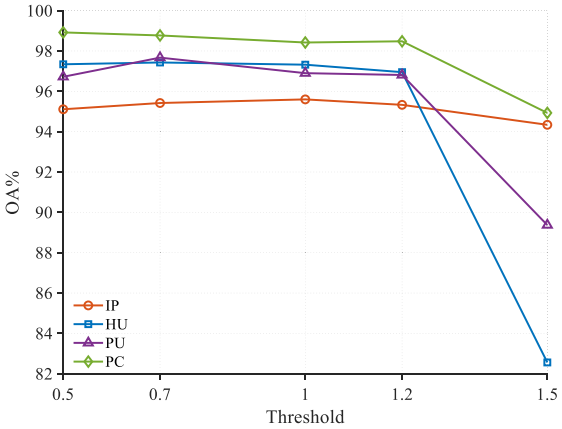


Fig. 8. Impact of neuronal threshold on OA for various hyperspectral datasets.

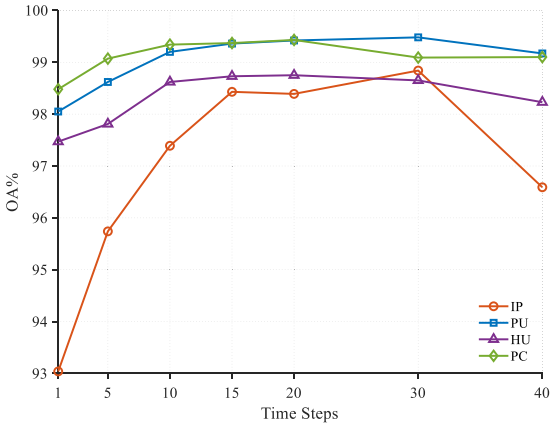


Fig. 9. Temporal dynamics of OA across multiple hyperspectral datasets.

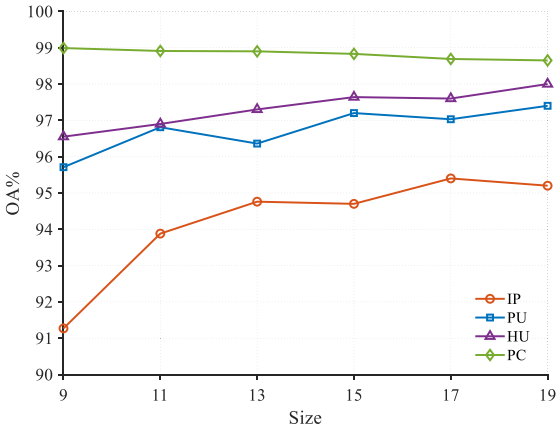


Fig. 10. Effect of input image size on OA across hyperspectral datasets.

For example, the Indian Pines and University of Houston datasets achieved 95.11% and 97.34% accuracy, respectively, with lower thresholds. In contrast, higher thresholds (1.2, 1.5) enable neurons to capture more salient features but may hinder information transfer.

2) Time Step Setting—Balancing Accuracy and Computational Load: In SNNs, the time step plays a crucial role. An appropriate time step helps maximize the capture of temporal dynamics while maintaining computational efficiency, thus improving classification accuracy as shown in Fig. 9. A balance is needed: too short a time step may fail to capture effective features, while too long a step may cause unnecessary computational overhead. Experimental results show that classification accuracy (OA) generally improves as the time step increases, with the most significant gains occurring in the early stages (1–15 steps). For example, the IP dataset shows a rapid OA increase between steps 1–15, reflecting its need for initial feature capture. The PU dataset improves continuously until reaching 99.43% OA at 30 steps, indicating the presence of complex temporal dynamics that require longer response times. The HU dataset achieves its optimal OA (98.95%) at 25 steps, while the PC dataset shows stable performance between 15 and 25 steps. These differences highlight the varying intrinsic feature complexities and model responses across the datasets.

3) Spatial Dimensionality—Effects on Feature Extraction and Model Complexity: It is crucial to investigate the effect of the spatial size of the input image on the classification performance; larger image sizes provide more spatial information but increase the computational burden. Different sizes ($9 \times 9, 11 \times 11, 13 \times 13, 15 \times 15, 17 \times 17, 19 \times 19$) were tested with a fixed optimal time step on four datasets. Experimental results shown in Fig. 10 that the proposed model's performance is significantly affected by spatial size. Smaller sizes (9×9, 11×11) offer higher computational efficiency but often fail to capture sufficient spatial context, especially in complex scenes. For example, the IP dataset's overall accuracy (OA) at 9×9 is 91.27%, much lower than at larger sizes. Conversely, larger sizes (17×17, 19×19) provide rich spatial context, improving feature capture and spatial relationships. The HU dataset achieves 97.8% OA at 19×19, an improvement of 1.25% points over 9×9. The PU dataset shows a steady OA increase from 95.71% at 9×9 to 97.2% at 19×19, suggesting that moderately increasing spatial size enhances performance. The PC dataset performs well at all sizes, achieving 98.99% OA even at 9×9, indicating a simpler feature distribution or low spatial correlation.

4) Comparison Between SSA and VSA: The experimental results, as shown in Fig. 11, demonstrate that the SSA mechanism consistently outperforms the VSA across multiple datasets. Particularly in the PU and PC datasets, SSA maintains higher OA at all time steps, with the PC dataset achieving near 100% accuracy throughout. In contrast, VSA

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 11:04:01 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
CLASSIFICATION ACCURACY OF VARIOUS METHODS IN OA, AA, κ, AND ACCURACY OF EACH CLASS ON IP DATASET

Class	SVM	ELM	EMP	2D-CNN	3D-CNN	SpectralFormer	SSFTT	SNN	**Proposed**
1	69.23	**100.00**	66.67	99.63	89.89	**100.0**	99.32	**100.0**	**100.0**
2	76.15	78.29	58.51	87.39	75.11	71.16	95.37	97.44	**97.82**
3	72.73	63.52	82.74	97.83	90.18	90.27	96.71	97.80	**98.32**
4	48.03	37.50	78.56	99.73	98.59	97.81	**100.0**	99.32	99.52
5	86.81	93.52	91.61	**98.14**	88.65	91.91	97.26	93.73	95.30
6	95.35	95.84	99.07	**99.82**	97.80	96.87	98.76	**99.74**	99.52
7	57.14	83.33	99.78	**100.0**	99.87	**100.0**	**100.0**	**100.0**	**100.0**
8	99.47	98.95	95.24	**100.0**	100.0	99.21	99.17	99.85	**99.95**
9	40.00	60.00	99.82	99.12	**100.0**	**100.0**	**100.0**	**100.0**	**100.0**
10	62.55	61.54	82.68	95.37	82.73	81.42	**97.07**	93.81	96.44
11	85.66	86.63	61.32	98.27	67.23	75.52	93.25	95.07	**99.46**
12	71.61	68.93	61.46	93.42	85.44	87.42	97.46	99.39	**99.60**
13	99.03	95.45	98.10	98.12	99.82	**100.0**	99.91	99.65	99.21
14	96.80	97.83	86.61	98.34	98.08	92.10	98.63	**99.31**	99.23
15	59.95	74.49	81.12	99.22	99.59	92.31	99.82	99.65	**100.0**
16	66.67	73.91	**100.0**	**100.0**	99.13	99.99	99.81	**100.0**	**100.0**
OA (%)	83.00	78.52	97.18	94.17	84.18	97.57	90.35	98.51	**98.54**
AA (%)	84.93	76.45	96.92	86.73	76.15	97.70	81.46	94.63	**99.10**
κ	80.54	75.39	96.78	93.35	81.94	97.24	89.06	97.18	**98.75**

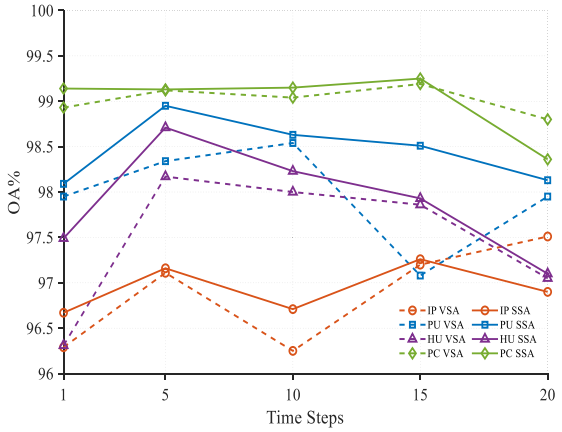


Fig. 11. Comparison of OA: SSA versus VSA Across different datasets.

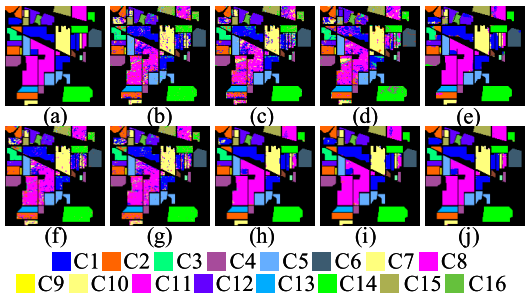


Fig. 12. Classification maps obtained by the compared methods on the IP dataset. (a) Reference map. (b) SVM. (c) ELM. (d) EMP. (e) 2D-CNN. (f) 3D-CNN. (g) SpectralFormer. (h) SSFTT. (i) SNN. (j) Proposed.

shows greater fluctuations across different time steps, especially in the IP and PU datasets. Notably, SSA achieves near-optimal classification accuracy even with fewer time steps (e.g., 5 steps), indicating its effectiveness under low computational complexity and shorter time frames.

Overall, SSA exhibits greater stability and accuracy across various datasets. Its ability to capture complex features and long-range dependencies allows it to achieve higher classification performance in fewer time steps. Additionally, it reduces computational complexity, highlighting its advantages in energy efficiency and faster inference.

IV. COMPARISON WITH OTHER METHODS

A. Experimental Design

1) *Conventional Methods:* We compared our method with SVM [7], extended morphological profile (EMP) [48], and extreme learning machine (ELM) [49]. SVM used an RBF kernel, ELM optimized only the output layer, and EMP applied dilation and erosion for spatial feature capture.
2) *CNN Backbone Networks:* We used 2D-CNN and 3D-CNN [50]. The 2D-CNN consisted of six convolutional layers and two max-pooling layers per module, followed by average pooling and a softmax layer. The 3D-CNN had a similar structure but with 3-D convolutions.
3) *Transformer Architectures:* We used SpectralFormer [17] and SSFTT [16]. SpectralFormer, based on Vision Transformer (ViT), applied attention and feedforward networks to segmented image patches, while SSFTT used 3-D and 2-D convolutions for feature extraction, followed by adaptive tokenization and Transformer processing.
4) *SNN:* We used a spike neural network [43] with an encoder, standard and depth-separable convolutional layers, and average pooling. The two-branch structure combined standard convolution and depth-pointwise convolution, followed by feature fusion, channel shuffling, and further convolution and pooling.

B. Evaluation of Experimental Results

In this section, we evaluated the classification performance of the compared methods using accuracy metrics and visualizations. Results for the IP, HU, PU, and PC datasets are presented in Tables VI–IX and Figs. 12–15. Each experiment

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 11:04:01 UTC from IEEE Xplore. Restrictions apply.

TABLE VII

CLASSIFICATION ACCURACY OF VARIOUS METHODS IN OA, AA, κ, AND ACCURACY OF EACH CLASS ON HU DATASET

Class	SVM	ELM	EMP	2D-CNN	3D-CNN	SpectralFormer	SSFTT	SNN	**Proposed**
1	96.21	97.87	94.07	97.84	94.68	98.96	97.61	98.35	**99.92**
2	98.41	97.25	98.01	**99.44**	90.83	98.80	98.78	97.74	99.42
3	99.95	100.0	98.49	**100.0**	99.21	99.86	99.66	99.33	99.67
4	97.92	97.17	98.69	97.91	93.83	93.89	99.73	99.91	**99.93**
5	98.34	89.55	98.65	**100.0**	94.23	98.79	**100.0**	**100.0**	**100.0**
6	99.08	**100.0**	98.67	97.23	93.19	99.38	**100.0**	99.23	99.83
7	95.12	70.69	51.89	95.35	95.17	93.93	68.93	**98.89**	97.69
8	90.54	90.94	91.26	91.00	79.94	89.63	**97.78**	95.97	96.07
9	84.12	65.79	61.73	90.58	89.07	93.61	**97.53**	94.70	94.81
10	92.62	71.85	92.72	99.92	94.36	97.56	98.94	98.49	**100.0**
11	90.04	67.57	91.10	97.33	92.95	93.52	98.66	99.12	**99.74**
12	87.48	86.07	78.55	98.54	81.62	93.03	99.20	97.71	**99.29**
13	71.76	65.90	83.74	98.51	99.77	80.81	97.70	**100.0**	**100.0**
14	93.71	90.30	98.17	99.92	**100.0**	99.07	99.96	**100.0**	99.65
15	99.47	99.82	98.93	**100.0**	97.90	99.81	99.82	**100.0**	**100.0**
OA(%)	93.19	83.74	86.94	97.23	91.82	95.36	96.06	98.65	**98.95**
AA(%)	92.95	86.05	83.48	97.58	92.98	95.39	96.69	98.68	**99.11**
κ	92.62	82.39	85.88	97.00	91.14	94.38	95.73	98.22	**98.80**

TABLE VIII

CLASSIFICATION ACCURACIES OF THE COMPARED METHODS IN TERMS OF OA, AA, κ, AND THE ACCURACIES OF EACH CLASS FOR THE PU DATASET

Class	SVM	ELM	EMP	2D-CNN	3D-CNN	SpectralFormer	SSFTT	SNN	**Proposed**
1	95.96	98.13	98.97	96.93	90.56	94.69	98.45	99.60	**99.99**
2	93.35	97.65	98.23	99.38	96.67	95.02	99.02	99.83	**100.0**
3	53.21	65.31	**99.32**	97.95	93.38	93.47	98.68	98.63	**99.32**
4	93.41	89.98	98.92	99.22	97.85	97.58	**99.40**	97.28	97.80
5	95.11	**100.0**	99.74	99.99	**100.0**	**100.0**	99.38	**100.0**	**100.0**
6	80.98	74.33	99.15	97.23	86.62	84.47	99.69	**99.96**	99.75
7	50.17	56.66	10.00	99.13	96.99	96.39	**100.0**	99.91	**100.0**
8	81.58	80.66	98.97	98.54	95.74	90.33	97.90	98.74	**100.0**
9	99.90	**100.0**	99.87	**100.0**	**100.0**	99.06	99.97	99.97	94.81
OA(%)	85.83	88.48	98.74	98.89	94.57	95.41	99.06	99.13	**99.24**
AA(%)	82.63	84.75	99.24	98.97	95.31	95.00	96.69	98.68	**99.34**
κ	81.30	84.82	98.31	98.53	92.82	93.95	98.74	**99.29**	99.14

TABLE IX

CLASSIFICATION ACCURACIES OF THE COMPARED METHODS IN TERMS OF OA, AA, κ, AND THE ACCURACIES OF EACH CLASS FOR THE PC DATASET

	SVM	ELM	EMP	2D-CNN	3D-CNN	SpectralFormer	SSFTT	SNN	**Proposed**
1	99.96	**100.0**	99.92	**100.0**	98.64	99.91	97.61	98.35	99.99
2	97.45	**98.98**	98.01	96.17	97.30	96.02	96.02	96.05	97.91
3	83.65	89.17	95.29	98.28	91.88	89.61	98.36	95.71	**98.70**
4	68.02	69.80	99.55	98.62	94.41	96.08	99.72	**99.81**	99.31
5	98.32	98.46	99.59	99.76	83.87	88.34	98.67	98.07	**99.99**
6	93.89	91.26	98.85	98.23	96.93	95.10	99.12	97.67	**99.87**
7	90.84	98.32	96.35	96.35	90.04	94.09	**99.28**	97.06	98.37
8	99.33	**99.80**	99.34	99.78	89.64	99.26	99.33	99.56	99.49
9	99.02	**100.0**	**100.0**	99.97	99.58	99.58	99.71	97.97	99.28
OA(%)	96.73	97.92	97.19	98.38	94.59	98.14	98.78	99.20	**99.51**
AA(%)	92.39	97.12	98.34	98.58	95.31	95.34	97.98	97.97	**99.27**
κ	95.37	96.40	98.91	98.12	92.42	97.36	98.90	98.71	**99.45**

was repeated five times to ensure fairness, and we report the average OA, average accuracy (AA), Kappa statistic, and category-wise accuracy.

The first experiment was conducted on the IP dataset using 100 samples per class for training, with the rest for validation. Fig. 12 illustrates the results. SVM, ELM, and EMP produced significant noise due to difficulty handling a large number of training samples [see Fig. 12(b)–(d)]. 2D-CNN, 3D-CNN, and SpectralFormer also showed noise, while SpectralFormer, SSFTT, and SNN performed better overall. However, some noise remained [see Fig. 12(g)–(i)]. Our proposed method formed a clearer classification map [Fig. 12(j)]. Table VI presents the classification results, showing that our method achieves the best OA, AA, and Kappa values, outperforming SSFTT and SNN.

The second experiment on the HU dataset used 100 samples per class for training, with the rest for validation. Fig. 13 shows large structures like tennis courts, parking lots, and running tracks, highlighting the effectiveness of the proposed method in maintaining structural integrity. As shown in Table VII, our method achieved the best classification performance, with very high accuracy across several categories.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 11:04:01 UTC from IEEE Xplore. Restrictions apply.

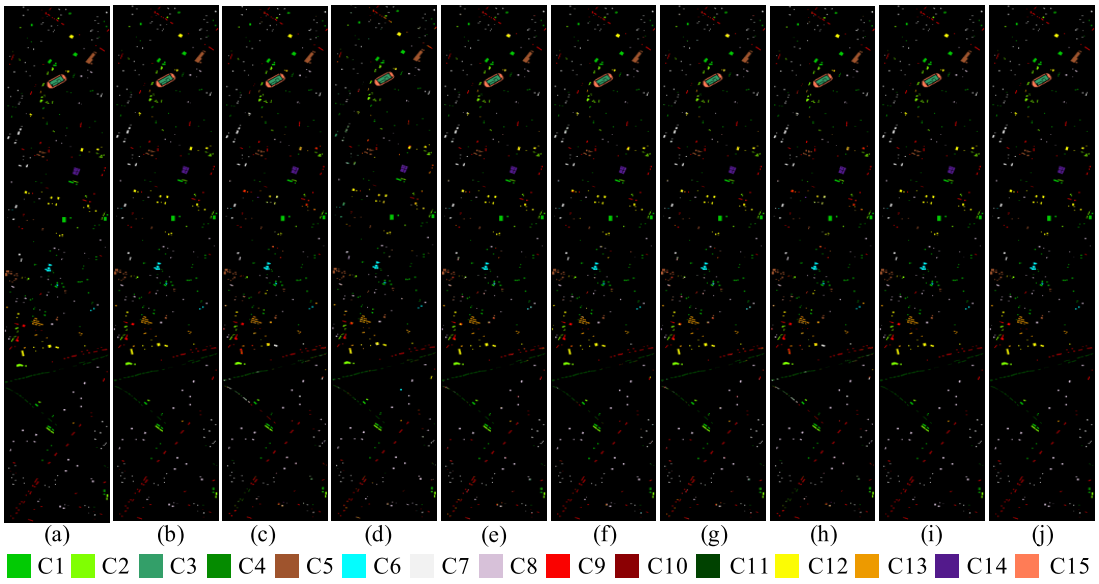


Fig. 13. Classification accuracies of the compared methods for the HU dataset. (a) Reference map. (b) SVM. (c) ELM. (d) EMP. (e) 2D-CNN. (f) 3D-CNN. (g) SpectralFormer. (h) SSFTT. (i) SNN. (j) Proposed.

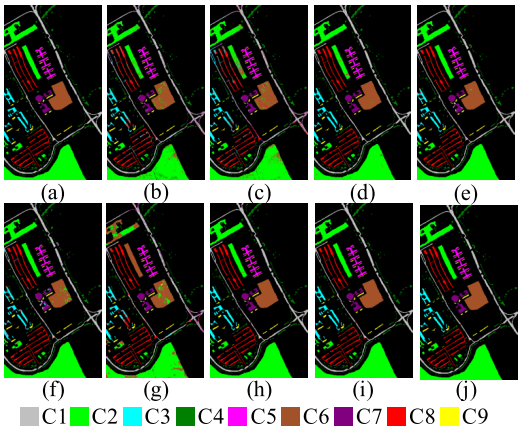


Fig. 14. Classification accuracies of the compared methods for the PU dataset. (a) Reference map. (b) SVM. (c) ELM. (d) EMP. (e) 2D-CNN. (f) 3D-CNN. (g) SpectralFormer. (h) SSFTT. (i) SNN. (j) Proposed.

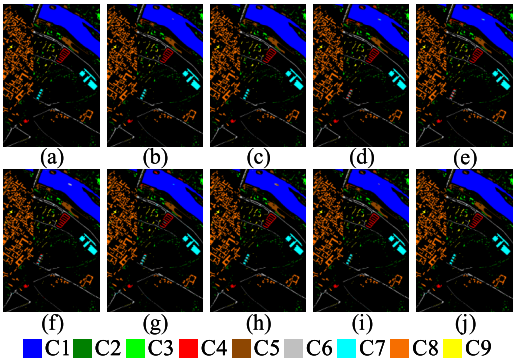


Fig. 15. Classification accuracies of the compared methods for the PU dataset. (a) Reference map. (b) SVM. (c) ELM. (d) EMP. (e) 2D-CNN. (f) 3D-CNN. (g) SpectralFormer. (h) SSFTT. (i) SNN. (j) Proposed.

It outperformed other methods in OA, AA, and Kappa metrics, showing more precise detail processing compared to SNN. The proposed method consistently maintained high accuracy across categories, demonstrating excellent feature extraction and classification capabilities. Fig. 13(i) shows clear boundaries with minimal noise or misclassification. In contrast, traditional methods like SVM, ELM, and EMP [Fig. 13(b)–(d)] and advanced deep learning methods like 2D-CNN and 3D-CNN [Fig. 13(e) and (f)] struggled with noise and unstable classification in complex scenes.

In the third experiment on the PU dataset, 100 samples per class were used for training, with the remainder for validation. Classification maps and accuracies are shown in Fig. 14 and Table VIII. The proposed method excels in preserving feature boundaries and reducing noise, outperforming traditional methods like SVM and ELM, which exhibit more noise and misclassifications [see Fig. 14(b) and (c)]. While deep learning methods like 2D-CNN, 3D-CNN, and SpectralFormer show improvement [see Fig. 14(e)–(g)], the proposed method [Fig. 14(j)] offers superior detail preservation and category differentiation. The proposed method achieved 100% accuracy in several categories, highlighting its robust classification capabilities. For challenging categories like category 3 (grass), it reached 99.32% accuracy, comparable to EMP and better than others. In category 6 (bare soil), it achieved 99.75% accuracy, slightly below SNN's 99.96%, but still superior to most methods. Overall, the proposed method effectively handles complex features, demonstrating its strength in HSI classification.

The final experiment was conducted on the PC dataset, using 100 samples from each class for training, with the remaining labeled samples used for validation. As shown in Fig. 15, the proposed method delivers high-quality classification image maps, particularly in terms of edges and textures, outperforming other methods. In Table IX, it is evident that

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 11:04:01 UTC from IEEE Xplore. Restrictions apply.

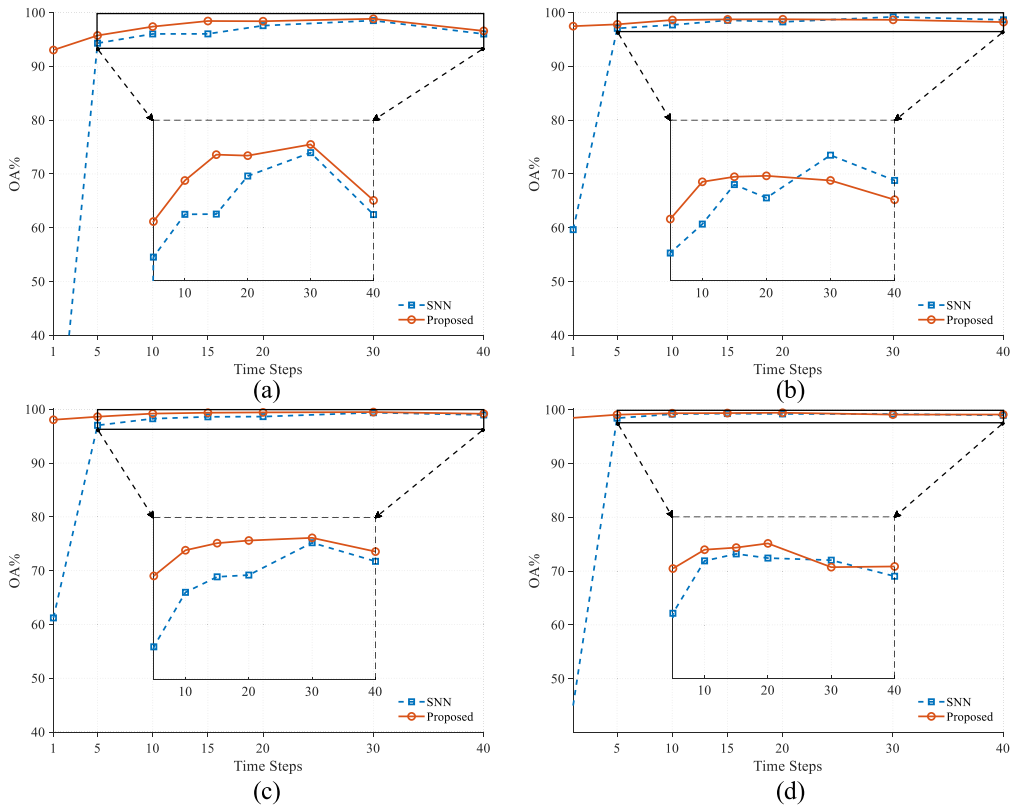


Fig. 16. Comparative performance analysis of SNN and proposed model on hyperspectral datasets over time steps. (a) IP. (b) HU. (c) PU. (d) PC.

deep learning-based methods struggle to perform well with limited training samples in certain challenging categories, such as water bodies, crosswalks, and unpaved parking lots. In contrast, the proposed method demonstrates superior performance, effectively addressing these challenges and outperforming the other methods in OA.

C. Comparison With Other SNNs

To rigorously evaluate the performance of our proposed method in HSI classification, we conducted a detailed comparison with a traditional SNN as a benchmark. This comparison focused on key aspects such as classification accuracy, computational efficiency, and energy consumption. All models underwent the same data preprocessing, training strategies, and evaluation metrics to ensure fairness. The primary metric was OA. This analysis aims to demonstrate the superior classification accuracy of our method while highlighting its advantages in efficiency and energy consumption, emphasizing its practicality for remote sensing applications.

1) Comparison With Same Step Size: The results show that our proposed method outperforms the traditional SNN across all metrics in HSI classification. For example, at 1 step and across four datasets, our method demonstrates superior effectiveness. In the IP dataset, it achieves a peak OA of 98.54% at 20 steps, significantly earlier than the SNN, which peaks at 98.51% after 30 steps, as shown in Fig. 16.

In the HU dataset, our method attains the highest OA of 98.95% at 25 steps, surpassing the SNN's 98.65% at 30 steps, indicating superior handling of medium-complexity features [Fig. 16(b)]. Similarly, in the PU dataset, it achieves an OA of 99.24% at 30 steps, slightly better than the SNN's 99.13%, showcasing consistent stability and efficiency [Fig. 16(c)]. In the PC dataset, our method reaches an OA of 99.48% at 30 steps, significantly outperforming the SNN's 98.96% at 40 steps, demonstrating rapid accuracy improvements and sustained performance [Fig. 16(d)].

2) Computation Time Analysis: Extensive comparisons of training times and step sizes were conducted across four datasets to evaluate the performance of the proposed method at various time steps, with traditional SNN models as benchmarks. As shown in Table X, the proposed method consistently achieved shorter training and testing times, particularly at the 1-step size, where it exceeded 95% performance across all datasets. Additionally, the proposed method met or surpassed the performance metrics of the traditional SNN [43] on nearly all datasets and time steps, demonstrating its robustness. The implementation of the SSA mechanism enhanced information utilization during multistep training, resulting in faster training and superior test performance compared to the traditional SNN.

V. Energy Consumption Analysis

In CNNs, each operation typically involves a floating-point multiplication and addition. In contrast, SNNs primarily perform additions using binary pulse signals. A key advantage is that neurons not firing impulses do not contribute

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 11:04:01 UTC from IEEE Xplore. Restrictions apply.

TABLE X
TRAINING AND TESTING TIMES OF SNN AND PROPOSED METHODS ACROSS VARIOUS HYPERSPECTRAL DATASETS AT DIFFERENT TIME STEPS

Dataset	Steps	SNN		Proposed	
		Training(s)	Test(s)	Training(s)	Test(s)
IP	1	35.23	1.28	**33.41**	**1.14**
	5	132.59	3.99	**132.10**	**3.16**
	10	247.67	7.43	**244.45**	**9.32**
	15	359.98	10.97	**351.66**	**10.35**
	20	469.77	14.95	**458.95**	**12.42**
	30	611.63	20.73	**609.00**	**20.44**
	40	**798.46**	**25.55**	835.56	32.46
HU	1	48.13	**1.76**	**37.04**	1.86
	5	140.32	7.15	**138.3**	**7.76**
	10	284.01	13.43	**281.16**	**12.57**
	15	401.18	**19.32**	**398.56**	20.41
	20	**479.95**	**26.83**	51.37	26.89
	30	**699.30**	**32.15**	732.61	38.41
	40	**914.49**	**41.24**	954.12	49.63
PU	1	25.25	6.11	**25.11**	**5.89**
	5	87.15	19.27	**85.49**	**18.91**
	10	**154.86**	**33.78**	156.21	36.49
	15	**233.3**	**49.42**	252.69	50.84
	20	**302.03**	**65.98**	316.42	83.79
	30	**416.38**	**95.22**	450.4	122.51
	40	**546.31**	**128.48**	572.61	153.88
PC	1	25.32	17.85	**24.90**	**13.88**
	5	95.43	66.62	**83.39**	**64.63**
	10	173.66	122.86	**163.2**	**122.55**
	15	**230.52**	**181.25**	227.5	239.15
	20	**299.76**	**239.65**	305.79	296.67
	30	**435.31**	**414.69**	440.56	421.29
	40	**579.43**	**513.58**	588.9	542.51

to computations, significantly reducing energy consumption, particularly in neuromorphic hardware. In CNNs, each operation typically involves high-energy-consuming floating-point multiplications and additions. In contrast, SNNs operate fundamentally differently, utilizing low-energy binary pulse signals where computations primarily involve additions. A key advantage of SNNs lies in their efficient utilization of energy, especially beneficial in neuromorphic hardware, where power efficiency is crucial.

In this study, the energy efficiency of the proposed method was evaluated using the PU dataset, employing energy estimation techniques detailed in [51]. Compared to CNNs, which consume approximately 4.6 picojoules (pJ) per 32-bit multiply-add (MAC) operation [52], SNNs require only about 0.9 pJ for an addition operation, marking a significant reduction of approximately 5.1× in energy consumption.

Fig. 17 illustrates the average pulse emission rate per layer based on pulse count, without canceling effects of positive and negative pulses. The energy consumption ratio is calculated based on the emission rate of each layer, as outlined in [51]. This approach underscores how SNNs leverage sparse, event-driven processing to minimize unnecessary computations and optimize energy efficiency, particularly advantageous in hardware implementations where power constraints are critical

$$E_{\text{CNN}} = \text{OP}_{\text{CNN}} \times E_{\text{MAC}} \tag{22}$$

$$E_{\text{SNN}} = \text{OP}_{\text{SNN}} \times E_{\text{AC}} \tag{23}$$

$$= \text{Spike rate} \times \text{OP}_{\text{CNN}} \times E_{\text{AC}} \tag{24}$$

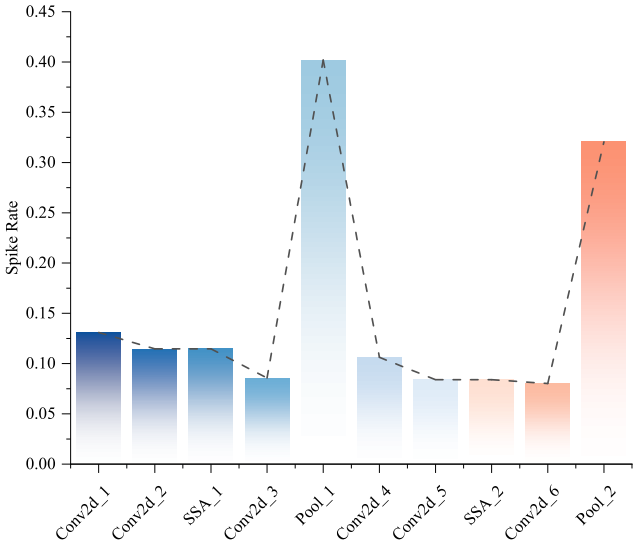


Fig. 17. Analysis of neuronal spiking rates across various layers in the PU dataset.

where E denotes the energy consumed, OP_{SNN} represents the number of MAC operations in the SNN, and OP_{AC} indicates the number of accumulation operations.

Based on the formula and energy consumption data, it is evident that implementing SNNs in hardware offers substantial energy-saving advantages. Specifically, the SSA mechanism demonstrates significantly lower energy consumption compared to the traditional attention mechanism (VSA), thereby enhancing overall model energy efficiency. Our study reveals that the total energy consumption of our model is only about 8.38% of that of a traditional ANN with the same architecture. Furthermore, our SSA-based SNN achieves nearly 15% greater efficiency compared to traditional SNN implementations [43].

Additionally, our method achieves superior results even within a single time step, underscoring its efficacy in real-time applications. This underscores the potential of SSA-based SNNs not only in reducing energy overhead but also in enhancing computational efficiency and performance in various tasks.

VI. CONCLUSION AND FUTURE WORK

In this work, we present a novel NN based on LIF neurons with direct coding. Our key contribution is a reparameterized focusing convolutional layer for enhanced spectral–spatial feature extraction, coupled with a spatial self-attention mechanism to model complex dependencies. Validated on four hyperspectral datasets, the method achieves high accuracy with fewer time steps, making it ideal for resource-constrained edge devices. Future research will optimize neuron models, improve spatio-temporal encoding, and extend to tasks like target detection and semantic segmentation, leveraging SNNs for hyperspectral analysis.

REFERENCES

[1] W. Li, Q. Du, and B. Zhang, "Combined sparse and collaborative representation for hyperspectral target detection," *Pattern Recognit.*, vol. 48, no. 12, pp. 3904–3916, Dec. 2015.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 11:04:01 UTC from IEEE Xplore. Restrictions apply.

[2] X. Yang and Y. Yu, "Estimating soil salinity under various moisture conditions: An experimental study," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 5, pp. 2525–2533, May 2017.

[3] D. Hong, N. Yokoya, J. Chanussot, and X. X. Zhu, "An augmented linear mixing model to address spectral variability for hyperspectral unmixing," *IEEE Trans. Image Process.*, vol. 28, no. 4, pp. 1923–1938, Apr. 2018.

[4] W. Xie, T. Jiang, Y. Li, X. Jia, and J. Lei, "Structure tensor and guided filtering-based algorithm for hyperspectral anomaly detection," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 7, pp. 4218–4230, Jul. 2019.

[5] J. A. Richards and X. Jia, "Using suitable neighbors to augment the training set in hyperspectral maximum likelihood classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 5, no. 4, pp. 774–777, Oct. 2008.

[6] J. Peng, L. Li, and Y. Y. Tang, "Maximum likelihood estimation-based joint sparse representation for the classification of hyperspectral remote sensing images," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 6, pp. 1790–1802, Jun. 2019.

[7] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 8, pp. 1778–1790, Aug. 2004.

[8] J. Ham, Y. Chen, M. M. Crawford, and J. Ghosh, "Investigation of the random forest framework for classification of hyperspectral data," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 3, pp. 492–501, Mar. 2005.

[9] S. Prasad and L. M. Bruce, "Limitations of principal components analysis for hyperspectral target recognition," *IEEE Geosci. Remote Sens. Lett.*, vol. 5, no. 4, pp. 625–629, Oct. 2008.

[10] T. V. Bandos, L. Bruzzone, and G. Camps-Valls, "Classification of hyperspectral images with regularized linear discriminant analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 47, no. 3, pp. 862–873, Mar. 2009.

[11] Y. Zhan, D. Hu, Y. Wang, and X. Yu, "Semisupervised hyperspectral image classification based on generative adversarial networks," *IEEE Geosci. Remote Sens. Lett.*, vol. 15, no. 2, pp. 212–216, Feb. 2018.

[12] L. Zhu, Y. Chen, P. Ghamisi, and J. A. Benediktsson, "Generative adversarial networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 9, pp. 5046–5063, Sep. 2018.

[13] X. Liao, B. Tu, J. Li, and A. Plaza, "Class-wise graph embedding-based active learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5522813.

[14] B. Tu, X. Liao, Q. Li, Y. Peng, and A. Plaza, "Local semantic feature aggregation-based transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5536115.

[15] B. Tu, W. He, Q. Li, Y. Peng, and A. Plaza, "A new context-aware framework for defending against adversarial attacks in hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5505114.

[16] L. Sun, G. Zhao, Y. Zheng, and Z. Wu, "Spectral-spatial feature tokenization transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5522214.

[17] D. Hong et al., "SpectralFormer: Rethinking hyperspectral image classification with transformers," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5518615.

[18] X. Huang, M. Dong, J. Li, and X. Guo, "A 3D-swin transformer-based hierarchical contrastive learning method for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5411415.

[19] Y. Yazid, I. Ez-Zazi, A. Guerrero-González, A. E. Oualkadi, and M. Arioua, "UAV-enabled mobile edge-computing for IoT based on AI: A comprehensive review," *Drones*, vol. 5, no. 4, p. 148, 2021.

[20] S. Ghosh-Dastidar and H. Adeli, "Spiking neural networks," *Int. J. Neural Syst.*, vol. 19, no. 4, pp. 295–308, Aug. 2009.

[21] J. He et al., "The development of spiking neural network: A review," in *Proc. IEEE Int. Conf. Robot. Biomim. (ROBIO)*, Dec. 2022, pp. 385–390.

[22] A. Safa et al., "Improving the accuracy of spiking neural networks for radar gesture recognition through preprocessing," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 6, pp. 2869–2881, Jun. 2021.

[23] B. Yin, F. Corradi, and S. M. Bohté, "Effective and efficient computation with multiple-timescale spiking recurrent neural networks," in *Proc. Int. Conf. Neuromorphic Syst.*, Jul. 2020, pp. 1–8.

[24] J. Stuijt, M. Sifalakis, A. Yousefzadeh, and F. Corradi, "μbrain: An event-driven and fully synthesizable architecture for spiking neural networks," *Frontiers Neurosci.*, vol. 15, May 2021, Art. no. 664208.

[25] J. Lee, H. Jeong, and S. Kang, "Derivative and GA-based methods in metamodeling of back-propagation neural networks for constrained approximate optimization," *Struct. Multidisciplinary Optim.*, vol. 35, no. 1, pp. 29–40, Nov. 2007.

[26] T. C. Wunderlich and C. Pehle, "Event-based backpropagation can compute exact gradients for spiking neural networks," *Sci. Rep.*, vol. 11, no. 1, p. 12829, Jun. 2021.

[27] W. Zhang and P. Li, "Spike-train level backpropagation for training deep recurrent spiking neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2019, pp. 1–11.

[28] S. R. Kulkarni and B. Rajendran, "Spiking neural networks for handwritten digit recognition—Supervised learning and network optimization," *Neural Netw.*, vol. 103, pp. 118–127, Jul. 2018.

[29] B. Tolooshams, S. Dey, and D. Ba, "Deep residual autoencoders for expectation maximization-inspired dictionary learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 6, pp. 2415–2429, Jun. 2021.

[30] S. Wang et al., "A progressive surrogate gradient learning for memristive spiking neural network," *Chin. Phys. B*, vol. 32, no. 6, May 2023, Art. no. 068704.

[31] J. Wu, Y. Chua, M. Zhang, G. Li, H. Li, and K. C. Tan, "A tandem learning rule for effective training and rapid inference of deep spiking neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 1, pp. 446–460, Jan. 2023.

[32] T. Chen, L. Wang, J. Li, S. Duan, and T. Huang, "Improving spiking neural network with frequency adaptation for image classification," *IEEE Trans. Cognit. Develop. Syst.*, vol. 16, no. 3, pp. 864–876, Jun. 2024.

[33] F. Xing, Y. Yuan, H. Huo, and T. Fang, "Homeostasis-based cnn-to-snn conversion of inception and residual architectures," in *Proc. Int. Conf. Neural Inf. Process.*, 2019, pp. 173–184.

[34] Y. Zhang, H. Xu, L. Huang, and C. Chen, "A storage-efficient SNN–CNN hybrid network with RRAM-implemented weights for traffic signs recognition," *Eng. Appl. Artif. Intell.*, vol. 123, Aug. 2023, Art. no. 106232.

[35] P. Plagwitz, F. Hannig, J. Teich, and O. Keszocze, "To spike or not to spike? A quantitative comparison of SNN and CNN FPGA implementations," 2023, *arXiv:2306.12742*.

[36] P. Plagwitz, F. Hannig, J. Teich, and O. Keszocze, "SNN vs. CNN implementations on fpgas: An empirical evaluation," in *Proc. Int. Symp. Appl. Reconfig. Comput.* Cham, Switzerland: Springer, 2024, pp. 3–18.

[37] Z. Zhou et al., "Spikformer: When spiking neural network meets transformer," 2022, *arXiv:2209.15425*.

[38] M. Yao et al., "Spike-driven transformer," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 1–13.

[39] Z. Wang, Y. Fang, J. Cao, Q. Zhang, Z. Wang, and R. Xu, "Masked spiking transformer," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2023, pp. 1761–1771.

[40] K. You et al., "SpikeZIP-TF: Conversion is all you need for transformer-based SNN," 2024, *arXiv:2406.03470*.

[41] J. C. Thiele, O. Bichler, and A. Dupret, "SpikeGrad: An ANN-equivalent computation model for implementing backpropagation with spikes," 2019, *arXiv:1906.00851*.

[42] A. Kasiński and F. Ponulak, "Comparison of supervised learning methods for spike time coding in spiking neural networks," *Int. J. Appl. Math. Comput. Sci.*, vol. 16, no. 1, pp. 101–113, Jan. 2006.

[43] Y. Liu, K. Cao, R. Li, H. Zhang, and L. Zhou, "Hyperspectral image classification of brain-inspired spiking neural network based on approximate derivative algorithm," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 6009405.

[44] M. Nelson and J. Rinzel, "The Hodgkin–Huxley model," in *The Book of Genesis*, vol. 2. New York, NY, USA: Springer, 1995.

[45] E. M. Izhikevich, "Simple model of spiking neurons," *IEEE Trans. Neural Netw.*, vol. 14, no. 6, pp. 1569–1572, Nov. 2003.

[46] D. Tal and E. L. Schwartz, "Computing with the leaky integrate-and-fire neuron: Logarithmic computation and multiplication," *Neural Comput.*, vol. 9, no. 2, pp. 305–318, 1997.

[47] Y. Kim, H. Park, A. Moitra, A. Bhattacharjee, Y. Venkatesha, and P. Panda, "Rate coding or direct coding: Which one is better for accurate, robust, and energy-efficient spiking neural networks?" in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2022, pp. 71–75.

[48] B. Kumar and O. Dikshit, "Hyperspectral image classification based on morphological profiles and decision fusion," *Int. J. Remote Sens.*, vol. 38, no. 20, pp. 5830–5854, Oct. 2017.

[49] D. B. Heras, F. Argüello, and P. Quesada-Barriuso, "Exploring ELM-based spatial–spectral classification of hyperspectral images," *Int. J. Remote Sens.*, vol. 35, no. 2, pp. 401–423, Jan. 2014.

[50] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3D–2D CNN feature hierarchy for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 2, pp. 277–281, Feb. 2019.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 11:04:01 UTC from IEEE Xplore. Restrictions apply.

[51] Y. Guo, Y. Liu, A. Oerlemans, S. Lao, S. Wu, and M. S. Lew, "Deep learning for visual understanding: A review," *Neurocomputing*, vol. 187, pp. 27–48, Apr. 2016.

[52] M. Horowitz, "1.1 computing's energy problem (and what we can do about it)," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2014, pp. 10–14.



Heng Li (Graduate Student Member, IEEE) received the bachelor's degree in communication engineering from Nanjing University of Information Science and Technology, Nanjing, China, in June 2023, where he is currently pursuing the M.Eng. degree in optoelectronic information engineering.

His research interests include hyperspectral image processing, deep learning, and spiking neural networks.



Bing Tu (Senior Member, IEEE) received the M.S. degree in control science and engineering from Guilin University of Technology, Guilin, China, in 2009, and the Ph.D. degree in mechatronic engineering from Beijing University of Technology, Beijing, China, in 2013.

From 2015 to 2016, he was a Visiting Researcher with the Department of Computer Science and Engineering, University of Nevada, Reno, NV, USA, which is supported by China Scholarship Council. He is currently a Full Professor with Nanjing University of Information Science and Technology, Nanjing, China. His research interests include sparse representation, pattern recognition, and analysis in remote sensing.

Dr. Tu is an Associate Editor of IEEE JOURNAL OF SELECTED TOPICS IN APPLIED EARTH OBSERVATIONS AND REMOTE SENSING.



Bo Liu (Member, IEEE) received the B.S. and Ph.D. degrees in optical engineering from Beijing University of Posts and Telecommunications, Beijing, China, in 2008 and 2013, respectively.

He is currently a Professor with the School of Physics and Optoelectronics, Nanjing University of Information Science and Technology (NUIST), Nanjing, China. His research interests include all optical signal processing, radio over fiber, and broadband optical communication.



Jun Li (Fellow, IEEE) received the B.S. degree in geographic information systems from Hunan Normal University, Changsha, China, in 2004, the M.E. degree in remote sensing from Peking University, Beijing, China, in 2007, and the Ph.D. degree in electrical engineering from the Instituto de Telecomunicações, Instituto Superior Técnico (IST), Universidade Técnica de Lisboa, Lisbon, Portugal, in 2011.

From 2013 to 2021, she was a Full Professor with Sun Yat-sen University, Guangzhou, China. Since 2022, she has been with China University of Geosciences, Wuhan, China, as a Full Professor. She has received several prestigious funding grants at the national and international levels. She has authored more than 160 journal citation report (JCR) articles, 60 international conference papers, and a book chapter.

Dr. Li has been serving as the Editor-in-Chief for *Journal of Selected Topics in Applied Earth Observations and Remote Sensing* since 2021.



Antonio Plaza (Fellow, IEEE) received the M.Sc. and Ph.D. degrees in computer engineering from the Hyperspectral Computing Laboratory, Department of Technology of Computers and Communications, University of Extremadura, Cáceres, Spain, in 1999 and 2002, respectively.

He is currently the Head of the Hyperspectral Computing Laboratory, Department of Technology of Computers and Communications, University of Extremadura. He has authored more than 600 publications, including over 200 JCR journal articles (over 160 in IEEE journals), 23 book chapters, and around 300 peer-reviewed conference proceeding papers. His research interests include hyperspectral data processing and parallel computing of remote sensing data.

Dr. Plaza was a member of the Editorial Board of IEEE GEOSCIENCE AND REMOTE SENSING NEWSLETTER from 2011 to 2012 and *IEEE Geoscience and Remote Sensing Magazine* in 2013. He was also a member of the Steering Committee of IEEE JOURNAL OF SELECTED TOPICS IN APPLIED EARTH OBSERVATIONS AND REMOTE SENSING (JSTARS). He is a fellow of IEEE for contributions to hyperspectral data processing and parallel computing of Earth observation data. He was a recipient of the recognition of Best Reviewers of IEEE GEOSCIENCE AND REMOTE SENSING LETTERS in 2009 and the Best Reviewer of IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING in 2010, for which he has served as an Associate Editor from 2007 to 2012. He was also a recipient of the Best Column Award of *IEEE Signal Processing Magazine* in 2015, the 2013 Best Paper Award of the JSTARS journal, and the Most Highly Cited Paper from 2005 to 2010 in *Journal of Parallel and Distributed Computing*. He received best paper awards at the IEEE International Conference on Space Technology and the IEEE Symposium on Signal Processing and Information Technology. He has served as the Director of Education Activities for IEEE GEOSCIENCE AND REMOTE SENSING SOCIETY (GRSS) from 2011 to 2012 and the President of the Spanish Chapter of IEEE GRSS from 2012 to 2016. He has reviewed more than 500 papers for over 50 different journals. He has served as the Editor-in-Chief for IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING from 2013 to 2017. He has guest edited ten special issues on hyperspectral remote sensing for different journals. He is also an Associate Editor of IEEE ACCESS (receiving the recognition as an Outstanding Associate Editor of the journal in 2017). Additional information is available at http://www.umbc.edu/rssipl/people/aplaza.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 11:04:01 UTC from IEEE Xplore. Restrictions apply.